

Software design patterns

Introduction

What ⚡⚡



Design pattern: An *elegant reusable solution to a commonly recurring problem* within a *given context* in software design.

In software development, there are certain problems that recur in a certain context.

📦 Some examples of recurring design problems:

Design Context	Recurring Problem
Assembling a system that makes use of other existing systems implemented using different technologies	What is the best architecture?
UI needs to be updated when the data in the application backend changes	How to initiate an update to the UI when data changes without coupling the backend to the UI?

After repeated attempts at solving such problems, better solutions are discovered and refined over time. These solutions are known as design patterns, **a term popularized by the seminal book *Design Patterns: Elements of Reusable Object-Oriented Software* by the so-called "Gang of Four" (GoF)** written by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.

Format ⚡⚡

The common format to describe a pattern consists of the following components:

- **Context:** The situation or scenario where the design problem is encountered.
- **Problem:** The main difficulty to be resolved.
- **Solution:** The core of the solution. It is important to note that the solution presented only includes the most general details, which may need further refinement for a specific context.
- **Anti-patterns** (optional): Commonly used solutions, which are usually incorrect and/or inferior to the Design Pattern.
- **Consequences** (optional): Identifying the pros and cons of applying the pattern.
- **Other useful information** (optional): Code examples, known uses, other related patterns, etc.

Singleton pattern

What ⚡⚡

Context

Certain classes should have no more than just one instance (e.g. the main controller class of the system). These single instances are commonly known as *singletons*.

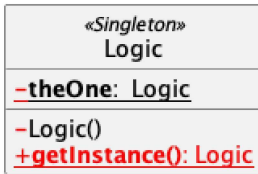
Problem

A normal class can be instantiated multiple times by invoking the constructor.

Solution

Make the constructor of the singleton class `private`, because a `public` constructor will allow others to instantiate the class at will. Provide a `public` class-level method to access the *single instance*.

Example:



The `<<Singleton>>` in the class above uses the UML *stereotype* notation, which is used to (optionally) indicate the purpose or the role played by a UML element. In this example, the class `Logic` is playing the role of a Singleton class. The general format is `<<role/purpose>>`.

Implementation ⚡

Here is the typical implementation of how the Singleton pattern is applied to a class:

```
1 class Logic {
2     private static Logic theOne = null;
3
4     private Logic() {
5         ...
6     }
7
8     public static Logic getInstance() {
9         if (theOne == null) {
10             theOne = new Logic();
11         }
12         return theOne;
13     }
14 }
```

Notes:

- The constructor is `private`, which prevents instantiation from outside the class.
- The single instance of the singleton class is maintained by a `private` class-level variable.
- Access to this object is provided by a `public` class-level operation `getInstance()` which instantiates a single copy of the singleton class when it is executed for the first time. Subsequent calls to this operation return the single instance of the class.

If `Logic` was not a Singleton class, a `Logic` object can be created as follows:

```
Logic m = new Logic();
```

But when it is a Singleton class, the single `Logic` object needs to be accessed as follows:

```
Logic m = Logic.getInstance();
```

Evaluation ⚡⚡⚡

Pros:

- easy to apply
- effective in achieving its goal with minimal extra work
- provides an easy way to access the singleton object from anywhere in the codebase

Cons:

- The singleton object acts like a global variable that increases coupling across the codebase.
- In testing, it is difficult to replace Singleton objects with stubs (static methods cannot be overridden).
- In testing, singleton objects carry data from one test to another even when you want each test to be independent of the others.

Given that there are some significant cons, it is recommended that you apply the Singleton pattern when, in addition to requiring only one instance of a class, there is a risk of creating multiple objects by mistake, and creating such multiple objects has real negative consequences.

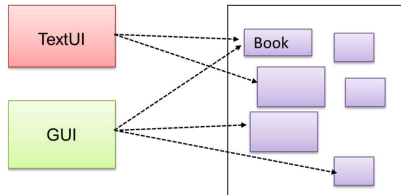
Facade pattern

What ⚡⚡⚡

Context

Components need to access functionality deep inside other components.

📦 The **UI** component of a **Library** system might want to access functionality of the **Book** class contained inside the **Logic** component.



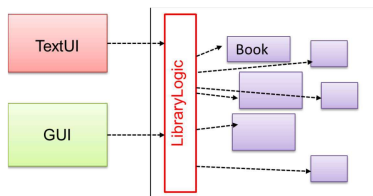
Problem

Access to the component should be allowed without exposing its internal details. e.g. the **UI** component should access the functionality of the **Logic** component without knowing that it contains a **Book** class within it.

Solution

Include a Facade class that sits between the component internals and users of the component such that all access to the component happens through the Facade class.

📦 The following class diagram applies the Facade pattern to the **Library System** example. The **LibraryLogic** class is the Facade class.



Command pattern

What ⚡⚡⚡

Context

A system is required to execute a number of commands, each doing a different task. For example, a system might have to support **Sort**, **List**, **Reset** commands.

Problem

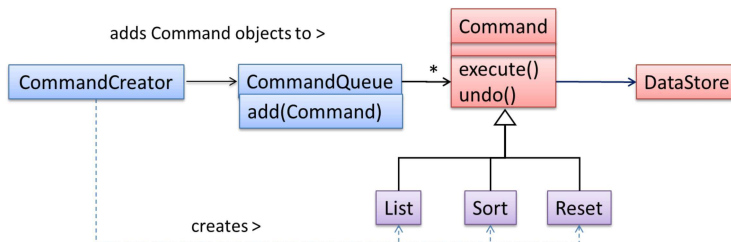
It is preferable that some part of the code executes these commands without having to know each command type. e.g., there can be a **CommandQueue** object that is responsible for queuing commands and executing them without knowledge of what each command does.

Solution

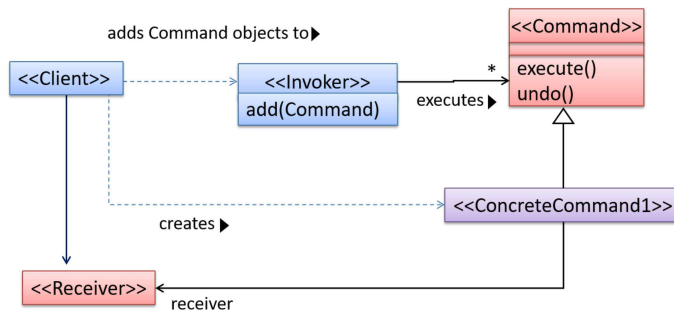
The essential element of this pattern is to have a general `<<Command>>` object that can be passed around, stored, executed, etc without knowing the type of command (i.e. via polymorphism).

Let us examine an example application of the pattern first:

📦 In the example solution below, the `CommandCreator` creates `List`, `Sort`, and `Reset` `Command` objects and adds them to the `CommandQueue` object. The `CommandQueue` object treats them all as `Command` objects and performs the execute/undo operation on each of them without knowledge of the specific `Command` type. When executed, each `Command` object will access the `DataStore` object to carry out its task. The `Command` class can also be an abstract class or an interface.



The general form of the solution is as follows.



The `<<Client>>` creates a `<<ConcreteCommand>>` object, and passes it to the `<<Invoker>>`. The `<<Invoker>>` object treats all commands as a general `<<Command>>` type. `<<Invoker>>` issues a request by calling `execute()` on the command. If a command is undoable, `<<ConcreteCommand>>` will store the state for undoing the command prior to invoking `execute()`. In addition, the `<<ConcreteCommand>>` object may have to be linked to any `<<Receiver>>` of the command (?) before it is passed to the `<<Invoker>>`. Note that an application of the command pattern does not have to follow the structure given above.

Model view controller (MVC) pattern

What ⚡

Context

Most applications support storage/retrieval of information, displaying of information to the user (often via multiple UIs having different formats), and changing stored information based on external inputs.

Problem

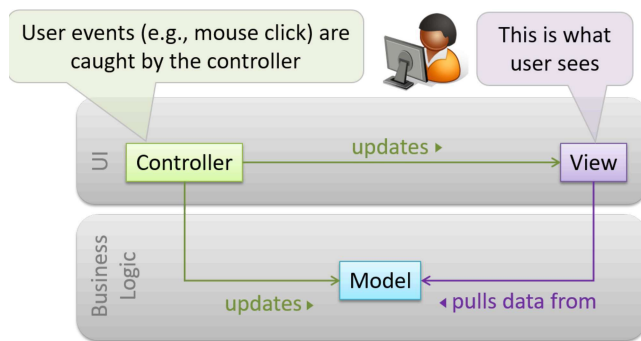
The high coupling that can result from the interlinked nature of the features described above.

Solution

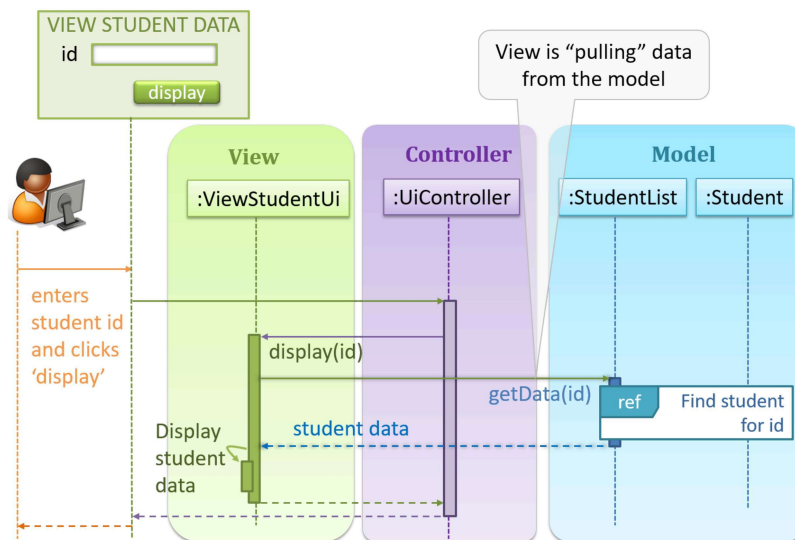
Decouple data, presentation, and control logic of an application by separating them into three different components: *Model*, *View* and *Controller*.

- *View*: Displays data, interacts with the user, and pulls data from the model if necessary.
- *Controller*: Detects UI events such as mouse clicks and button pushes, and takes follow up action. Updates/changes the model/view when necessary.
- *Model*: Stores and maintains data. Updates the view if necessary.

The relationship between the components can be observed in the diagram below. Typically, the UI is the combination of *View* and *Controller*.



Given below is a concrete example of MVC applied to a student management system. In this scenario, the user is retrieving the data of a student.



In the diagram above, when the user clicks on a button using the UI, the 'click' event is caught and handled by the `UiController`. The `ref` frame indicates that the interactions within that frame have been extracted out to another separate sequence diagram.

Note that in a simple UI where there's only one view, *Controller* and *View* can be combined as one class.

There are many variations of the MVC model used in different domains. For example, the one used in a desktop GUI could be different from the one used in a web application.

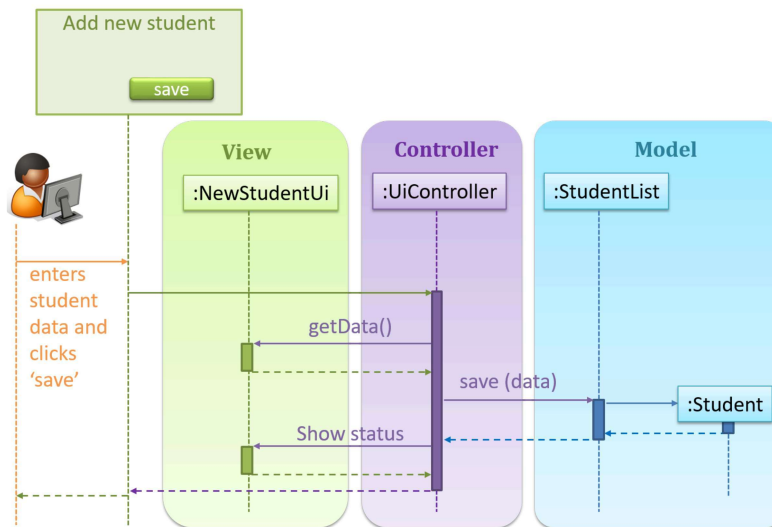
Observer pattern

What ⚡⚡⚡

Context

An object (possibly more than one) is interested in being notified when a change happens to another object. That is, some objects want to 'observe' another object.

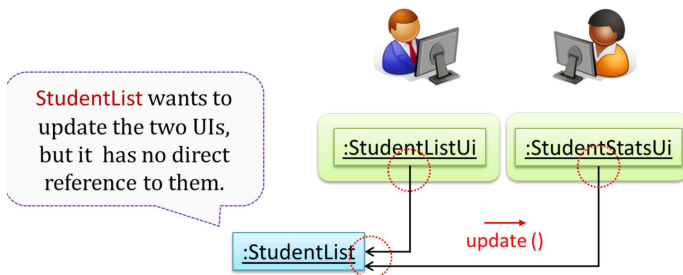
Consider this scenario from a student management system where the user is adding a new student to the system.



Now, assume the system has two additional views used in parallel by different users:

- **StudentListUi** : that accesses a list of students and
- **StudentStatsUi** : that generates statistics of current students.

When a student is added to the database using **NewStudentUi** shown above, both **StudentListUi** and **StudentStatsUi** should get updated automatically, as shown below.



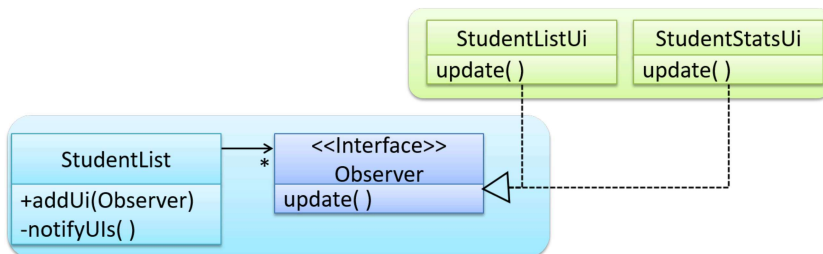
However, the **StudentList** object has no knowledge about **StudentListUi** and **StudentStatsUi** (note the direction of the navigability) and has no way to inform those objects. This is an example of the type of problem addressed by the Observer pattern.

Problem

The 'observed' object does not want to be coupled to objects that are 'observing' it.

Solution

Force the communication through an interface known to both parties.



Here is the Observer pattern applied to the student management system.

During the initialization of the system,

1. First, create the relevant objects.

```
1 StudentList studentList = new StudentList();
2 StudentListUi listUi = new StudentListUi();
3 StudentStatsUi statsUi = new StudentStatsUi();
```

2. Next, the two UIs indicate to the `StudentList` that they are interested in being updated whenever `StudentList` changes. This is also known as 'subscribing for updates'.

```
1 studentList.addUi(listUi);
2 studentList.addUi(statsUi);
```

3. Within the `addUi` operation of `StudentList`, all Observer object subscribers are added to an internal data structure called `observerList`.

```
1 // StudentList class
2 public void addUi(Observer o) {
3     observerList.add(o);
4 }
```

Now, whenever the data in `StudentList` changes (e.g. when a new student is added to the `StudentList`),

1. All interested observers are updated by calling the `notifyUIs` operation.

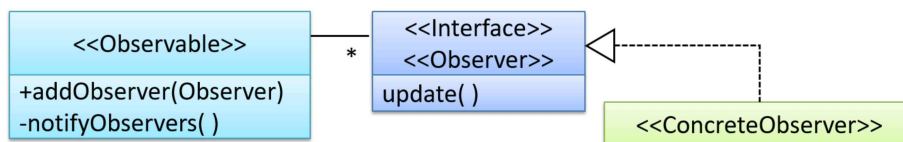
```
1 // StudentList class
2 public void notifyUIs() {
3     // for each observer in the list
4     for (Observer o: observerList) {
5         o.update();
6     }
7 }
```

2. UIs can then pull data from the `StudentList` whenever the `update` operation is called.

```
1 // StudentListUI class
2 public void update() {
3     // refresh UI by pulling data from StudentList
4 }
```

Note that `StudentList` is unaware of the exact nature of the two UIs but still manages to communicate with them via an intermediary.

Here is the generic description of the observer pattern:



- `<<Observer>>` is an interface: any class that implements it can observe an `<<Observable>>`. Any number of `<<Observer>>` objects can observe (i.e. listen to changes of) the `<<Observable>>` object.
- The `<<Observable>>` maintains a list of `<<Observer>>` objects. `addObserver(Observer)` operation adds a new `<<Observer>>` to the list of `<<Observer>>`s.
- Whenever there is a change in the `<<Observable>>`, the `notifyObservers()` operation is called that will call the `update()` operation of all `<<Observer>>`s in the list.

📦 In a GUI application, how is the Controller notified when the “save” button is clicked? UI frameworks such as JavaFX have inbuilt support for the Observer pattern.